05.06 2026

Building a Neural Network

By: Jonathon Tucker, Daniel Akl, Kamrin Bains

Goals

- Train on MNIST dataset
- Test model against MNIST data
  - ~10,000 test cases
- Test model against Class data
  - 290 testing digits
- Increase accuracy with each iteration

Dataset overview

- MNIST: 60k training images
- 28x28 grayscale
  - (perfect tests)
- Class dataset: 290 images, 29 per 0-9
  - Not-so-perfect

Model Architecture

- Two hidden layers
- 256 and 128
- Weights included in final report
- Originally used 3 hidden layers but swapped to two
  - Each hidden layer learns a new level of abstraction, curves, and edges
  - Presentation results are using two hidden layers

```
Sequential(
  (0): Linear(in_features=784, out_features=256, bias=True)
  (1): ReLU()
  (2): Linear(in_features=256, out_features=128, bias=True)
  (3): ReLU()
  (4): Linear(in_features=128, out_features=10, bias=True)
  (5): LogSoftmax(dim=1)
)

Number of hidden layers: 2
Neurons per hidden layer: [256, 128]
```

Training run 1

- 0.01 learning rate was first through our experiments
- Epoch size 5 in early experiments but ended on 15 (down from 20)

```
Accuracy on class dataset: 36.90% (107/290)

Per-digit accuracy:
  Digit 0: 3/29 (10.3%)
  Digit 1: 5/29 (17.2%)
  Digit 2: 22/29 (75.9%)
  Digit 3: 21/29 (72.4%)
  Digit 4: 9/29 (31.0%)
  Digit 5: 18/29 (62.1%)
  Digit 6: 8/29 (27.6%)
  Digit 7: 4/29 (13.8%)
  Digit 8: 12/29 (41.4%)
  Digit 9: 5/29 (17.2%)
```

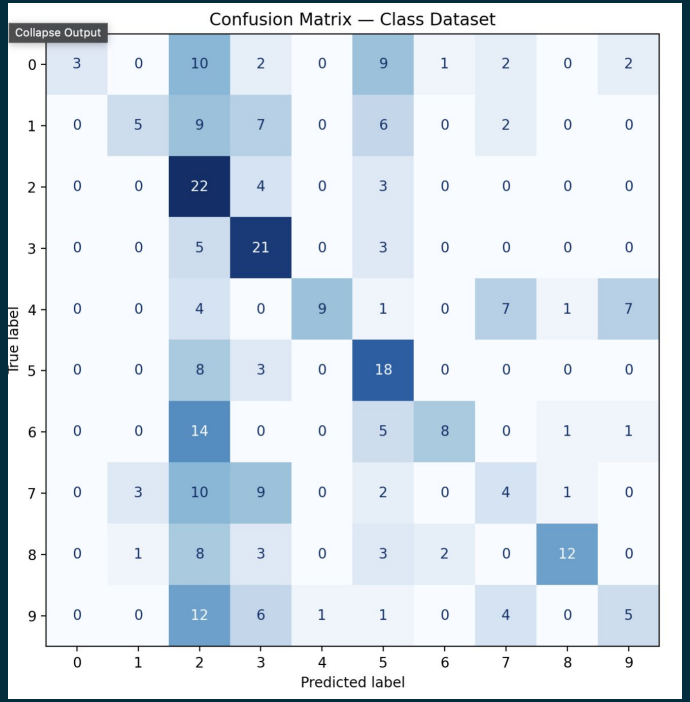
```python
# Training
loss_function = nn.NLLLoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)


epochs = 15
for epoch in range(epochs):
    for images, labels in train_loader:
        optimizer.zero_grad()
        images = images.view(images.shape[0], -1)
        output = model(images)
        loss = loss_function(output, labels)
        loss.backward()
        optimizer.step()
    print(f'Epoch [{epoch+1}/{epochs}], Loss: {loss.item():.4f}')
```

```
Epoch [1/15], Loss: 0.3691
Epoch [2/15], Loss: 0.1148
Epoch [3/15], Loss: 0.0600
Epoch [4/15], Loss: 0.2750
Epoch [5/15], Loss: 0.0110
Epoch [6/15], Loss: 0.2275
Epoch [7/15], Loss: 0.4912
Epoch [8/15], Loss: 0.1587
Epoch [9/15], Loss: 0.0438
Epoch [10/15], Loss: 0.2339
Epoch [11/15], Loss: 0.5638
Epoch [12/15], Loss: 0.0758
Epoch [13/15], Loss: 0.1071
Epoch [14/15], Loss: 0.4726
Epoch [15/15], Loss: 0.2702
```

Confusion Matrix run 1

- LR: 0.01
- Results are not great
- Predicts 2 & 3 fairly accurately
  - This is due to it primarily guessing 2 despite the #
- Not reliable



Training run 2

- 0.001 learning rate was best through our experiments
- Epoch size 5 in early experiments but ended on 15
- Nearly 14% from the learning rate adjustment

```python
# Training
loss_function = nn.NLLLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)


epochs = 15
for epoch in range(epochs):
    for images, labels in train_loader:
        optimizer.zero_grad()
        images = images.view(images.shape[0], -1)
        output = model(images)
        loss = loss_function(output, labels)
        loss.backward()
        optimizer.step()
    print(f'Epoch [{epoch+1}/{epochs}], Loss: {loss.item():.4f}')
```

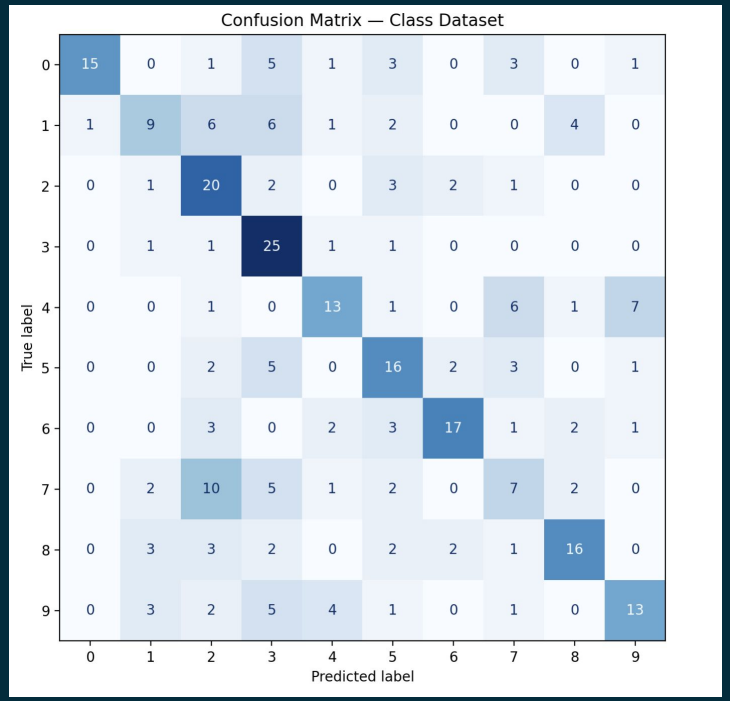
```
Epoch [1/15], Loss: 0.3001
Epoch [2/15], Loss: 0.0368
Epoch [3/15], Loss: 0.1592
Epoch [4/15], Loss: 0.0184
Epoch [5/15], Loss: 0.0503
Epoch [6/15], Loss: 0.0255
Epoch [7/15], Loss: 0.2085
Epoch [8/15], Loss: 0.0010
Epoch [9/15], Loss: 0.1176
Epoch [10/15], Loss: 0.0074
Epoch [11/15], Loss: 0.0508
Epoch [12/15], Loss: 0.1658
Epoch [13/15], Loss: 0.0415
Epoch [14/15], Loss: 0.0675
Epoch [15/15], Loss: 0.2875
```

```
Accuracy on class dataset: 52.07% (151/290)

Per-digit accuracy:
  Digit 0: 15/29 (51.7%)
  Digit 1: 9/29 (31.0%)
  Digit 2: 20/29 (69.0%)
  Digit 3: 25/29 (86.2%)
  Digit 4: 13/29 (44.8%)
  Digit 5: 16/29 (55.2%)
  Digit 6: 17/29 (58.6%)
  Digit 7: 7/29 (24.1%)
  Digit 8: 16/29 (55.2%)
  Digit 9: 13/29 (44.8%)
```

Confusion Matrix run 2

- Final iteration; good diagonal
- Mainly struggles with 1 & 7
  - Also has issues with 4 & 9
- Far more reliable than test #1



Training Run #3

- 0.001 learning rate was best through our experiments
- Epoch size 30

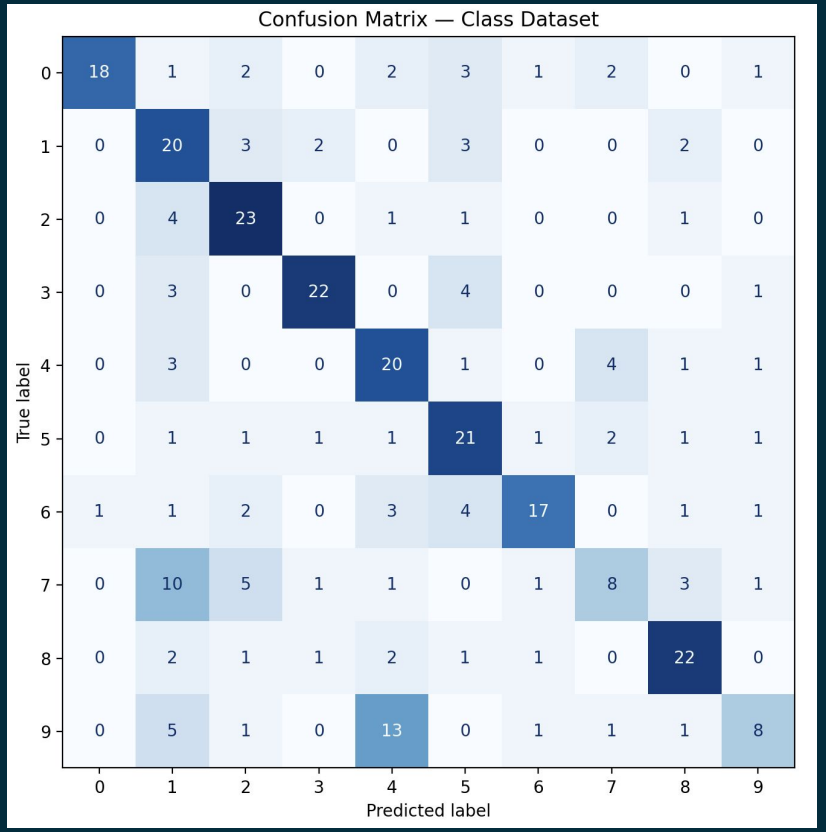
```
Accuracy on class dataset: 59.67% (179/300)

Per-digit accuracy:
  Digit 0: 18/30 (60.0%)
  Digit 1: 20/30 (66.7%)
  Digit 2: 23/30 (76.7%)
  Digit 3: 22/30 (73.3%)
  Digit 4: 20/30 (66.7%)
  Digit 5: 21/30 (70.0%)
  Digit 6: 17/30 (56.7%)
  Digit 7: 8/30 (26.7%)
  Digit 8: 22/30 (73.3%)
  Digit 9: 8/30 (26.7%)
```

```
Epoch [1/30], Loss: 0.1249
Epoch [2/30], Loss: 0.0596
Epoch [3/30], Loss: 0.0106
Epoch [4/30], Loss: 0.1868
Epoch [5/30], Loss: 0.0741
Epoch [6/30], Loss: 0.0116
Epoch [7/30], Loss: 0.0012
Epoch [8/30], Loss: 0.1336
Epoch [9/30], Loss: 0.1842
Epoch [10/30], Loss: 0.0040
Epoch [11/30], Loss: 0.0291
Epoch [12/30], Loss: 0.0043
Epoch [13/30], Loss: 0.0152
Epoch [14/30], Loss: 0.0656
Epoch [15/30], Loss: 0.0242
Epoch [16/30], Loss: 0.1138
Epoch [17/30], Loss: 0.0253
Epoch [18/30], Loss: 0.0020
Epoch [19/30], Loss: 0.1363
Epoch [20/30], Loss: 0.0109
Epoch [21/30], Loss: 0.0028
Epoch [22/30], Loss: 0.0120
Epoch [23/30], Loss: 0.0083
Epoch [24/30], Loss: 0.0359
Epoch [25/30], Loss: 0.0167
Epoch [26/30], Loss: 0.0032
Epoch [27/30], Loss: 0.0427
Epoch [28/30], Loss: 0.0002
Epoch [29/30], Loss: 0.0529
Epoch [30/30], Loss: 0.0102
```

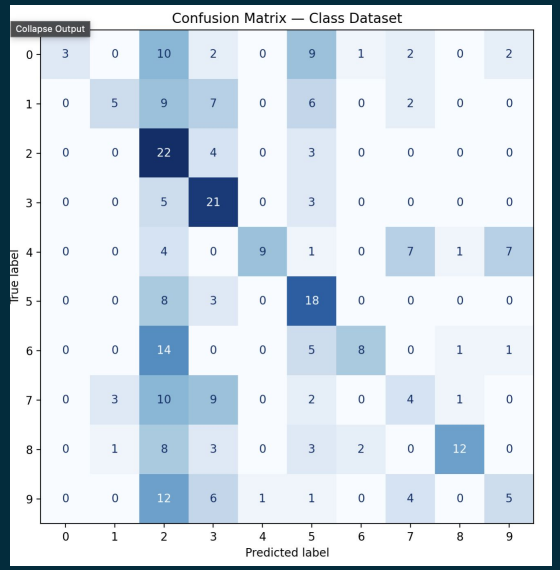
Confusion Matrix #3

- LR: 0.001
- Epochs: 30
- Best diagonal
  - Struggles most with 7 & 9

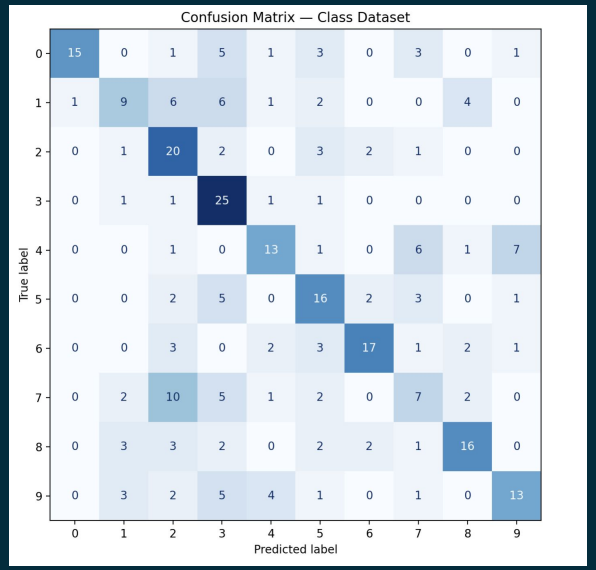


Confusion Matrix

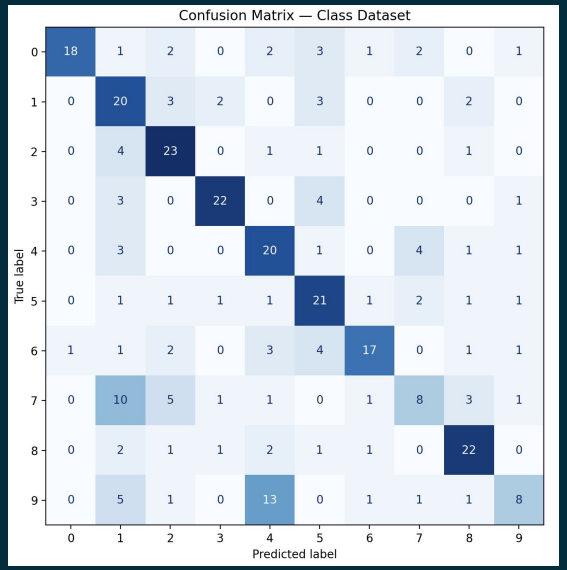
Run 1



Run 2



Run 3



Comparing MNIST Data

- Majority MNIST tests came out at 96%+ accuracy despite changes to hyperparameters

Run against MNIST set

```
test_set = torchvision.datasets.MNIST(root='~/.pytorch/MNIST_data/', train = False, dow
test_loader = torch.utils.data.DataLoader(test_set, batch_size=64, shuffle = False)
model.eval()
correct = 0
total = 0
with torch.no_grad():
     for images, labels in test_loader:
         images = images.view(images.shape[0], -1)
         output = model(images)
         _, predicted = torch.max(output, 1)
         correct += (predicted == labels).sum().item()
         total += labels.size(0)

print(f'Accuracy on MNIST test set: {100 * correct / total:.2f}% ({correct}/{total})')
```

```
Accuracy on MNIST test set: 97.69% (9769/10000)
```

Results

- MNIST v Our NN:
  - Greater than 96% with different hyperparameters
- Class v Our NN:
  - Last test ended with 59.67% against the class
- Increased Accuracy?:
  - With each experiment the accuracy increased (~36% → ~59%)